Lesson 11 Teacher's Key: Wave After Wave

Unity - Instructor Reference Only

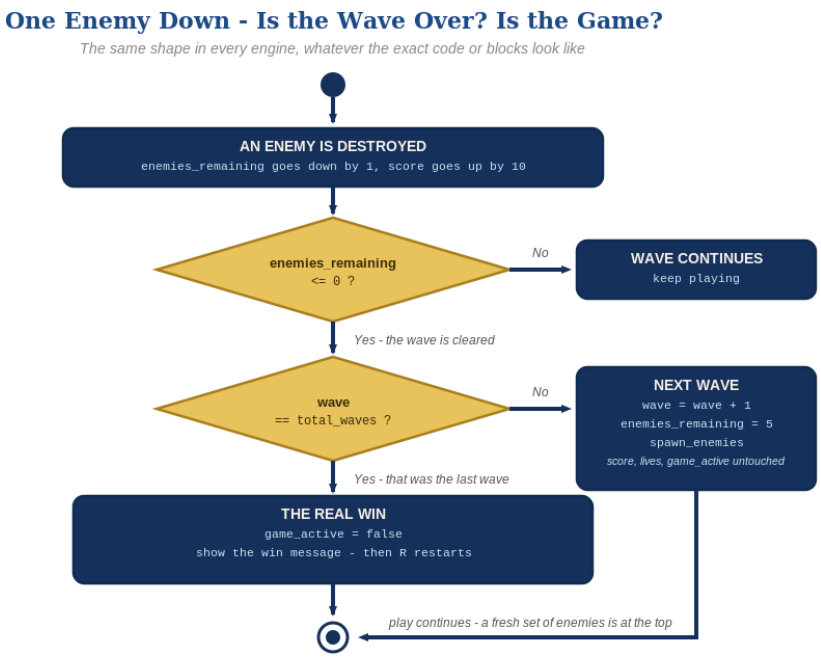
VIDEO GAME CREATION

What This Lesson Teaches

Reusing EnemySpawner.SpawnEnemies() from a new caller, driven by gameplay itself - and a concrete, honest look at what static members cost: a static method cannot use an Inspector reference, so a wave request has to be passed through a flag.

Level 1 - Full Worked Solution

- EnemySpawner is unchanged. SpawnEnemies() already has two callers (Start() and ResetGame()); GameManager's Update() becomes the third, when the wave check raises nextWaveRequested.
- wave, totalWaves and nextWaveRequested are static fields on GameManager, parallel to score and lives. HUD.cs reads wave and totalWaves every frame, the same poll-every-frame pattern as Lesson 9.
- The flag exists because RegisterKill() is static: it cannot use the Spawner slot (Inspector slots belong to one component instance). So it leaves a note and the instance Update() acts on it. This is the real cost of statics - the book edition revisits it (Book Notes: reconsider statics).



Clearing a wave asks a second question. If it was not the last wave, the game spawns a fresh set of enemies and carries on. The same shape in every engine.

Why This Structure

- No singleton and no Find...(): the one reference is the Inspector slot from Lesson 10, the simplest and most familiar way to connect two objects.
- The flag is deliberately visible and named for what it does. It is a small, honest workaround that sets up the Semester 2 conversation about instance versus static state.

Level 2 / Level 3 Talking Points

- Ask students to name every place SpawnEnemies() is now called from (EnemySpawner's Start(), ResetGame(), and GameManager's Update() on a wave request): three callers, one implementation.
- Why not call spawner.SpawnEnemies() straight from RegisterKill()? Let students try: the compiler complains about using an instance member from a static method. That is the lesson.
- For students comparing engines: Unity's flag-then-Update, Godot's call_deferred and Scratch's broadcast are three idioms for 'ask for a wave now, make it at a safe moment'.
- The Stalemate (Level 3): do NOT hand over the fix. It was Lesson 10's Level 3 and is deliberately left in so it becomes irritating enough that students go back and fix it. Waves make it worse: after any escape, enemies_remaining can never reach 0, so that wave never ends - at any of the three waves. Confirm it is real if a student reports it, name it (the Stalemate), and nudge: 'what has to be true for a wave to be over?' The worked fix below makes an escape also resolve an enemy, and routes both the kill path and the escape path through ONE wave-cleared decision. Accept any design that guarantees

every wave ends in exactly one outcome. Test: 3 kills and 2 escapes, 4 and 1, 0 and 5 (the last should be a loss only if lives run out), in each of the three waves.

Level 2 - complete code:

```
// Level 2 is verification only - the Level 1 code is already correct, so
no lines change.
// Checks: (1) clear all five three times: waves 1 and 2 continue, wave 3
shows the win message;
// (2) lose the last life in wave 2: lose message, no new wave;
// (3) R after a win and after a loss: wave 1, score 0, lives 3, exactly
five enemies.
```

Level 3 - complete code (additions on top of Level 1 highlighted):

Highlighted lines are the Level 3 additions on top of the Level 1 code. Methods shown are the final versions of everything Level 3 touches.

```
// GameManager.cs - Level 3 (final versions of everything Level 3
touches):
public static float waveMessageTimeLeft = 0f;

static void WaveCleared() {
    if (wave >= totalWaves) {
        hasWon = true;
        gameActive = false;
    } else {
        wave++;
        enemiesRemaining = 5;
        nextWaveRequested = true;
    }
}

public static void RegisterKill() {
    enemiesRemaining--;
    score += 10;
    if (enemiesRemaining <= 0) {
        WaveCleared();
    }
}

public static void RegisterEscape() {
    lives--;
    enemiesRemaining--;
    if (lives <= 0) {
        hasLost = true;
        gameActive = false;
    } else if (enemiesRemaining <= 0) {
        WaveCleared();
    }
}

public void ResetGame() {
    spawner.ClearEnemies();
    score = 0;
    lives = 3;
    wave = 1;
    enemiesRemaining = 5;
```

```
    hasWon = false;
    hasLost = false;
    nextWaveRequested = false;
    waveMessageTimeLeft = 0f;
    gameActive = true;
    spawner.SpawnEnemies();
}

void Update() {
    if (!gameActive && Keyboard.current.rKey.wasPressedThisFrame) {
        ResetGame();
    }
    if (nextWaveRequested) {
        nextWaveRequested = false;
        spawner.SpawnEnemies();
        waveMessageTimeLeft = 1f;
    }
    if (waveMessageTimeLeft > 0f) {
        waveMessageTimeLeft -= Time.deltaTime;
    }
}

void OnGUI() {
    // ...your existing You Win / Game Over labels stay exactly as they
are...
    if (waveMessageTimeLeft > 0f) {
        GUI.Label(new Rect(350, 150, 300, 40), "Wave " + wave + "!");
    }
}
```

Common Student Mistakes

- Game-state drift: wave or nextWaveRequested not reset in ResetGame(), so a restart after wave 3 begins at wave 3, or fires a stray wave. Static fields keep their values.
- Forgetting to drag WaveText onto the HUD's Wave Text slot: NullReferenceException in HUD.Update().
- Trying to call spawner.SpawnEnemies() from RegisterKill(): a static method cannot use the instance Spawner slot.
- Checking only enemiesRemaining <= 0 without the wave comparison, so clearing wave 1 immediately wins.
- A student reports a wave that never ends: that is the Stalemate. Do not hand over the fix.